Internet of Things 33 (2025) 101650

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot

Research article

A comprehensive and realistic performance evaluation of post-quantum security for consumer IoT devices

Yacoub Hanna [a,*], Jessica Bozhko [b], Samet Tonyali [c], Ricardo Harrilal-Parchment [a], Mumin Cebe [d], Kemal Akkaya [a]

[a] Department of Electrical and Computer Engineering, Advanced Wireless and Security Lab, Florida International University, Miami, FL 33174, United States of America
[b] Department of Computer Science, Auburn University, Auburn, AL 36849, United States of America
[c] Department of Software Engineering, Gumushane University, Gumushane, Turkey
[d] Department of Computer Science, Marquette University, Milwaukee, WI, 53233, United States of America

ARTICLE INFO

Keywords:
Post-quantum cryptography
Key encapsulation mechanism
TLS
IoT
IP over BLE
CRL
OCSP

ABSTRACT

The computational capacity envisaged for quantum computers poses a significant threat to today's traditional cryptographic algorithms. Although they are not yet large enough to compromise current cryptographic protocols, one can practice retrospective decryption since data packets traveling through the Internet can be easily sniffed. This threat extends to wireless communication security within consumer IoT devices that use lightweight cryptography due to limited computational power. Thus, countermeasures against potential quantum attacks should be preemptively adopted. NIST is leading efforts to standardize several algorithms as quantum-resistant Key Exchange Mechanisms (KEMs) and Digital Signatures. In this paper, we investigate the viability of these Post-Quantum algorithms in the Transport Layer Security (TLS) of power-constrained IoT devices. Specifically, it focuses on two widely used IoT network protocol stacks, i.e., Bluetooth Low Energy (BLE) and Wi-Fi. To this end, we build a realistic IoT testbed running IP over BLE. Our evaluation considers the impact of several realistic factors for the first time, such as using a chain of certificates on the server side and incorporating certificate validation methods such as Online Certificate Status Protocol (OCSP) and Certificate Revocation Lists (CRL). We also evaluate the impact of mutual authentication between the server and the client. Utilizing the outcomes of this evaluation, we then propose a novel approach for IoT devices to dynamically choose the most efficient KEM algorithm for TLS based on the device's physical network interface. The performance results provide valuable insights with respect to the TLS latency and energy consumption of consumer IoT devices.

1. Introduction

Within a few decades, quantum computers are envisaged to be accessible at least as a cloud service [1]. On the one hand, this will potentially provide many benefits to a list of application areas ranging from pharmaceutical research to financial modeling [2], but on the other hand, it poses a serious threat to the cryptographic primitives that are used to secure digital communications. Specifically, the Shor's algorithm [3] can solve the prime factorization problem and the discrete logarithm problem and its elliptic

* Corresponding author.
E-mail addresses: yhann002@fiu.edu (Y. Hanna), jab0245@auburn.edu (J. Bozhko), samet.tonyali@gumushane.edu.tr (S. Tonyali), rharr119@fiu.edu (R. Harrilal-Parchment), mumin.cebe@marquette.edu (M. Cebe), kakkaya@fiu.edu (K. Akkaya).

https://doi.org/10.1016/j.iot.2025.101650
Received 16 September 2024; Received in revised form 16 March 2025; Accepted 14 May 2025
Available online 16 June 2025

curve variant in polynomial time [4] in contrast with classical computers, which require sub-exponential time [5] at best regarding the security parameter. Hence, the RSA cryptosystem and the Diffie–Hellman key exchange protocol will be rendered useless when cryptographically-relevant quantum computers (CRQCs) are built. Therefore, the Shor's algorithm cannot be avoided because it completely eliminates the source of the hardness that makes these cryptographic algorithms secure.

CRQCs have the potential of enabling attackers to decrypt the encrypted network traffic and to forge digital signatures and certificates. Although this seems to be a problem that will be faced in the future the attackers can start capturing encrypted network traffic today and decrypt them after CRQS emerged out, which is a strategy called *harvest now, decrypt later*. To minimize such risks, in 2016, the National Institute of Standards and Technology (NIST) took action to standardize several public-key encryption, key encapsulation, and digital signature algorithms [6]. As of this writing, the NIST announced that CRYSTALS-Kyber has been standardized as Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) while CRYSTALS-Dilithium and SPHINCS+ have been standardized as Module-Lattice-Based Digital Signature Standard (ML-DSA) and Stateless Hash-Based Digital Signature Standard (SLH-DSA), respectively. Moreover, it was reported that Falcon will be standardized as Fast-Fourier Transform over NTRU-Lattice-Based Digital Signature Algorithm (FN-DSA) [7].

The Open Quantum Safe (OQS) project [8] has implemented these algorithms with the levels of security specified by the NIST [6]. However, the literature lacks of studies investigating the feasibility, performance, and overhead to be incurred by the applications employing them. One of such applications is the Internet of Things (IoT).

IoT devices typically generate some quantitative data about their surroundings and report them to remote servers via a gateway and by using the gateway's public IP address. However, the availability of vast number of IP addresses supported by IPv6 enables end-to-end connection and makes IoT devices reachable all over the world. Owing to RFC7668 - IPv6 over BLE [9], even consumer IoT devices with a BLE interface will be able to connect to any remote server directly. Nevertheless, a gateway will be needed to translate from BLE protocols to TCP/IP protocols. Therefore, instead of implementing a new layer in IoT devices' network stack, to use their storage resources efficiently, gateways can support BLE along with Wi-Fi and Ethernet. In the future, not only smart devices, but also classical Wi-Fi routers will be supporting BLE routing. Minew's G1 IoT gateway [10] can be given as an example of such routers.

IoT devices will encounter the same security risks as traditional computer systems as a result of their dependence on the same cryptographic protocols. In particular, protocol suites such as Transport Layer Security (TLS) and its UDP variant DTLS utilize RSA or ECC, both of which can be compromised by CRQCs. Hence, there is a crucial necessity to create and implement cryptographic protocols that are resistant to quantum attacks in order to secure IoT communications.

While some of the previous work studied the performance of PQ-TLS in general networks [11,12], these studies did not consider the overhead that comes from the incorporation of certificate revocation approaches such as OCSP or CRL.

Also, none of these studies considered the performance of PQ-TLS under a low-rate wireless protocol such as IP over BLE and its overhead on the energy resources of the involved IoT devices.

In this paper, we investigate the effectiveness of Post-Quantum Cryptography (PQC) algorithms in relation to consumer IoT devices during their connection through TLS, which functions as the predominant Internet protocol for ensuring secure communication across various platforms (e.g., HTTP). In this configuration, cryptography is utilized to agree on a symmetric key and to generate certificate signatures, and is used by IoT devices having one or more communication interfaces, such as BLE and Wi-Fi. To this end, we included a chain of certificates, implemented IP over BLE with the addition of a gateway, included the OCSP Stapling and CRLs into the certificate validation process, measured the energy consumption of IoT device, evaluated the non/cached OCSP Stapling, and designed a dynamic PQ algorithm suite selection mechanism for IoT devices.

The results of this study indicated that PQ KEM and digital signature performance are highly impacted by the underlying wireless connection, especially BLE. For instance, Dilithium2 can be a good choice for Wi-Fi, whereas Falcon512 is a more feasible digital signature algorithm for BLE. Moreover, Kyber512 is ideal for BLE when combined with Falcon512. Furthermore, with the addition of the certificate validation methods, there was an increase in the communication overhead, especially when using OCSP Stapling. Finally, we observed that the energy consumption due to varying PQ choices does not change on the IoT device, which offers flexibility.

Drawing on these outcomes, we propose and evaluate a dynamic switching mechanism for PQC within TLS, whereby an IoT consumer device chooses the KEM and digital signature algorithms that best fit its underlying MAC layer. The evaluation indicates that this dynamic approach offers advantages without bringing any side effects.

The rest of the paper is organized as follows: In the next section, we summarize the related work. Section 3 provides a detailed overview of the concepts that were utilized, while Section 5 presents three TLS configurations for the experiments. Section 6 introduces a new approach; dynamic switching for PQC. Section 8 presents details of the testbed that we used to perform the experiments. In Section 9, we present and discuss the experiment results. Lastly, Section 11 paves the way for further studies and concludes the paper.

2. Related work

This section summarizes the research efforts that investigated the application of PQC on software/hardware as well as networking studies.

2.1. Software/hardware PQ studies

Healthcare systems have recently become one of the most common IoT application areas. Thus, patients can be remotely monitored and tracked about their health status and medication, which is private information and should not be disclosed to unauthorized parties. To this end, Gupta et al. [13] proposed a lattice-based authentication and access control protocol. However, Adeli et al. [14] analyzed that protocol and revealed its vulnerabilities. Then, they proposed a new authentication and access control scheme based on the Saber. The formal and informal security of the proposed scheme was analyzed with respect to some well-known attacks. Also, performance evaluations were carried out on a Zynq Ultrascale FPGA. However, the proposed scheme was tested neither on a testbed nor in a simulator. Hence, validation and verification of digital certificates were out of the scope of the study.

Software updates are inevitable for sustainable security in IoT as well as in any other network. To this end, the IETF has specified a standard – software updates for Internet of Things (SUIT) – defining a security architecture for IoT software updates, standardizing format data and cryptographic primitives (e.g., hash functions and digital signatures) to assure that the update is legitimate. Banegas et al. [15] used the open-source implementation of SUIT in the RIOT operating system in order to evaluate and compare the performance of classical and quantum-resistant digital signature algorithms on several 32-bit IoT hardware such as ARM Cortex-M, RISC-V, and Espressif. As opposed to our work, this study focuses only on digital signatures and does not consider digital certificates at all. Besides, the performances are evaluated based only on the time elapsed for signing and verifying signatures and the size of data occupying space in stack or flash memory. Moreover, no network implications are considered in the experiments and the discussion.

2.2. PQ networking studies

There have been some research efforts on the performance of PQ algorithms when deployed within TLS. For instance, one of the initial performance evaluation studies is on the NIST's third round KEM and digital signature algorithms [16] assuming a wired networking setup. Similarly, Tasopoulos et al. [17,18] focused on the energy consumption of PQ algorithms on a NUCLEO-F439ZI development board. Finally, McLoughlin et al. [19] implemented and evaluated the performance of a quantum-resistant DTLS 1.3 handshake protocol for IoT devices. Moreover, recent research has explored the practical integration of post-quantum cryptographic primitives into real-world security protocols within constrained and delay-sensitive environments. In [20], the authors evaluate the impact of incorporating PQ key exchange and digital signatures into IKEv2/IPSec for satellite communications, highlighting latency and fragmentation challenges due to high packet loss and long propagation delays. Meanwhile, [21] presents the first realistic testbed-based integration of PQ-TLS into 5G control plane protocols, showing that although performance overhead exists, it remains tolerable and compatible with legacy systems. These studies provide insight into the challenges and trade-offs of deploying PQC in latency-sensitive IoT networks. However, none of these studies considered setups for low-bandwidth environments by incorporating certificate management, including CRLs and OCSPs.

In his Master's thesis [22], Sabanci built an ns-3 simulation environment that virtualizes an NB-IoT network on top of 5G infrastructure and investigated the computational and communication overhead when post-quantum key exchange and digital signature algorithms are used for securing NB-IoT applications. Their aim and network environment was different than ours.

In our previous work [23], we evaluated the efficiency of quantum-resistant algorithms in TLS handshake protocol for consumer IoT devices. However, that preliminary work did not consider certificate revocation issues and offered a very limited evaluation setup with a single self-signed server certificate. The analysis also did not include energy consumption. This extended work offers a realistic and comprehensive setup with CA, certificate revocation components, and the introduction of a new dynamic algorithm selection for KEMs. Different from all of the existing works, our work incorporates the validation and verification of the certificate chain, including CRLs and OCSP stapling during a TLS handshake when PQ algorithms are used. Moreover, we propose an approach for IoT devices to dynamically choose the convenient KEM and digital signature algorithms based on the device's physical interface, i.e., Wi-Fi or BLE.

2.3. Our contribution

A consumer IoT ecosystem consists of one or more IoT devices (e.g., smart watch, virtual personal assistant) and a gateway device such as a SOHO device that acts as an intermediary between local IoT network and the web services. These IoT devices typically have limited storage capacity and computational resources. As a consequence, the manufacturers prefer implementing weak security measures, or worse, providing no security at all. This is the major security challenge posed by consumer IoT ecosystems.

To prevent attackers from exploiting such vulnerabilities, cryptographic protocols providing acceptable security levels must be implemented so as to respect the limited resources of the IoT devices and to accommodate them for their principal operations.

As summarized in two subsections above, the existing literature lacks an end-to-end and holistic performance evaluation of PQC algorithms for consumer IoT ecosystems. A list of shortcomings detected in the literature is given below.

- Evaluations on an end-to-end realistic testbed (i.e., one with consumer IoT device, gateway, web server, and remote digital certificate management infrastructure)
- Evaluations of digital certificates and digital certificate validation and verification methods in terms of communication and computational overhead
- Flexibility of choosing KEM and digital signature algorithm pairs based on some criteria

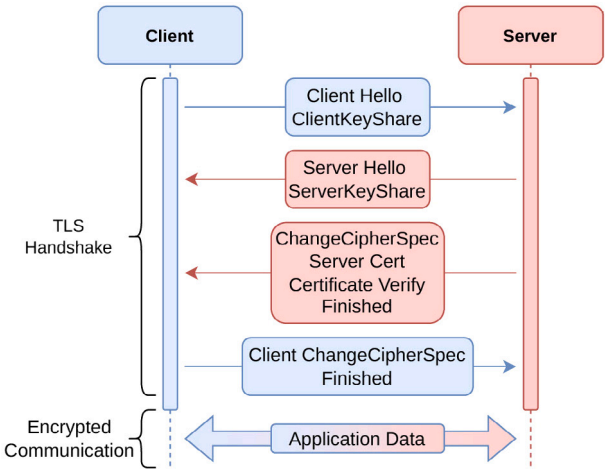

Fig. 1. TLS handshake messages.

This paper aims to bridge this gap in the existing literature by setting up a testbed consisting of two Raspberry Pis and a laptop acting as consumer IoT device, gateway, and web server, respectively (Please see Fig. 8). We also leased some resources from Google Cloud on which we built our digital certificate management infrastructure. Thus, we managed to measure the communication delay, energy consumption, unique memory footprint, and computational overhead experiments in a realistic ecosystem. Moreover, we evaluated the performance of PQC KEM and digital signature algorithms in TLS in the context of both one-way and mutual authentication. All these evaluations included different configurations of certificate validation and verification methods. Furthermore, we proposed a dynamic switching mechanism for PQC KEM and digital signature algorithms based on the application's communication interface type, power source, and latency requirements.

3. Preliminaries

In this section, we describe briefly the fundamental concepts and technologies we use in our work.

3.1. Transport layer security (TLS) handshake

The TLS handshake protocol is used to establish trust and agree upon a shared secret key, which is used in symmetric encryption operations between a server and a client machine. The steps of this protocol are illustrated in Fig. 1. The two methods that TLS supports are *Diffie–Hellman (DH) key exchange* and the *Key Encapsulation Mechanism (KEM)* for establishing the shared key. It should be noted that digital certificates are transferred in both scenarios in order to share public keys, as will be explained in more detail in the following subsection.

The hardness of the discrete logarithm problem is used to secure the DH key exchange protocol, in which two parties create a shared secret key (the session key) using randomly generated private numbers. Elliptic-Curve (EC) DH key exchange is one of the DH variants. Fig. 2 outlines the steps of ECDH key exchange. Alice and Bob want to establish a symmetric session key to encrypt and authenticate data traffic between them. Therefore, both parties should agree on an elliptic curve E that is defined over a finite field F_p and a base point G on the curve. Then, they pick a random private key as Alice's key d_A and Bob's key d_B. They perform an elliptic multiplication using G and the random private key to obtain a public key for each of Alice Q_A and Bob Q_B. After that, they exchange the computed public keys and perform another elliptic curve point multiplication using their own private key and each other's public key, which yields the same secret key sk.

Alternatively, the KEM method uses public-key encryption to exchange the session key between parties [8]. Fig. 3 illustrates how a KEM is utilized to exchange a secret key. In some scenarios, KEM could be related to digital certificates, as in the context of hybrid encryption schemes and key management. When asymmetric encryption is used for key encapsulation, the recipient's public key is usually obtained from a digital certificate. For example, in the figure, one of the participants, Bob, shares his SSL/TLS certificate with Alice. She could extract Bob's public key PK_B out of his certificate, generate a random session key k, compute the ciphertext, encapsulate key using Bob's public key, and send Bob the ciphertext. Bob decrypts the encrypted message using his private key SK_B and reveals the symmetric session key k.

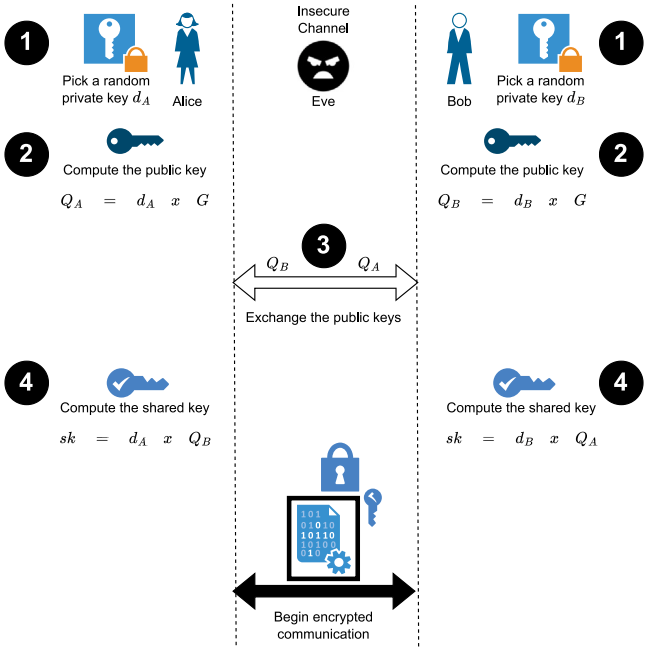

Fig. 2. Steps of Elliptic-Curve Diffie–Hellman key exchange.

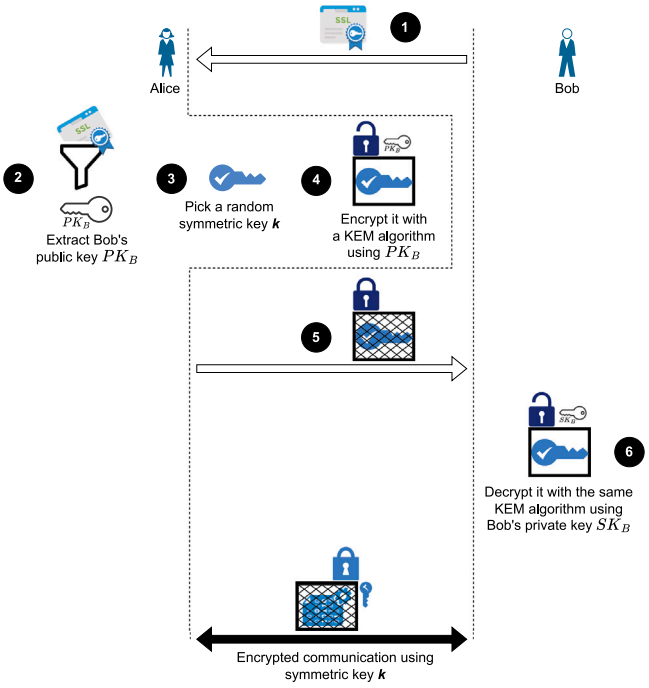

Fig. 3. Steps of Key Encapsulation Mechanism (KEM).

3.2. Public key certificates and mutual authentication in TLS

In order to prove the ownership of a public key to the recipient, the public key must be digitally signed and issued as a digital certificate by a trustworthy organization called a Certificate Authority (CA). In a classical setup (web browser and server), as shown

Table 1
Storage costs and security levels of post-quantum cryptographic algorithms.

Algorithms	Security level	Public key size (bytes)	Secret key size (bytes)	Signature/Ciphertext size (bytes)
Falcon512	1	897	1281	752 (Signature)
Falcon1024	5	1793	2305	1462 (Signature)
Dilithium2	2	1312	2528	2420 (Signature)
Dilithium3	3	1952	4000	3293 (Signature)
Dilithium5	5	2592	4864	4595 (Signature)
S+ SHA	1	32	64	17088 (Signature)
Kyber512	1	800	1632	768 (KEM)
Kyber1024	5	1568	3168	1568 (KEM)

in Fig. 1, only the server needs to transmit the client a certificate in order to verify its public key and identity under TLS. In cases where both the client and the server need to authenticate each other, they exchange their digital certificate, which is called *mutual authentication*.

3.3. Quantum-resistant algorithms

Shor's algorithm demonstrates that the widely-used discrete logarithm and integer factorization problems are relatively simple for a quantum computer to solve in the age of quantum computing [24]. In order to create public-key cryptosystems that are considered to be secure even against quantum computers, post-quantum cryptography is being developed [8]. These methods make use of a range of mathematical operations, such as learning with error problems, zero-knowledge proofs, and lattice problems.

The NIST announced that they have standardized Dilithium and SPHINCS+ digital signature algorithms and Kyber KEM [25]. Also, Falcon digital signature algorithm is envisioned to be standardized in the foreseeable future. Therefore, we evaluate these four algorithm families in this study.

- **CRYSTALS-Dilithium:** The Dilithium uses module lattice problems to provide quantum-resistant security [26]. Dilithium2 claims NIST level 2 security, Dilithium3 claims level 3 security, and Dilithium5 claims level 5 security.
- **Falcon:** The Falcon family of signature schemes provide security by using NTRU lattice problems [26]. Falcon-512 claims NIST level 1 security, and Falcon-1024 claims NIST level 5 security.
- **SPHINCS+:** The SPHINCS+ family of signature schemes uses hash-based signatures to provide post-quantum security [26]. Only the 128-bit level 1 security variants of SPHINCS+ SHA, and SHAKE were considered in this experiment due to their excessively large TLS handshake sizes and slow connection speed displayed over BLE. However, we felt it was important to include SPHINCS+ due to the recent announcement by NIST [25] deciding which post-quantum algorithms to use as a standard.
- **CRYSTALS-Kyber:** The Kyber family of KEM schemes use lattice-based cryptography algorithm [27]. Out of all the KEM schemes that were in the 3rd round of the NIST standardization competition, such as Kyber, NTRU, Classic McEliece, and Saber, NIST has chosen the Kyber family as the only KEM to be used. Thus, Kyber512, Kyber768, and Kyber1024 claim NIST levels 1, 3, and 5, respectively [25].

The OQS benchmarking data evaluate the performance of KEM algorithms by measuring the number of operations per second for key generation, encapsulation, and decapsulation processes. For example, the Kyber algorithm demonstrates efficient performance across different security levels, which reflects the trade-off between security level and computational efficiency. On the other hand, digital signatures have been analyzed for their computational performance as well. Falcon512 exhibits rapid key generation and signing times, making it suitable for applications requiring certain cryptographic operations. The computational cost benchmarking could be obtained by OQS testing which shows all the operations for both KEM and digital signatures [28].

The storage cost of PQC algorithms, including public key sizes, secret key sizes, signature sizes, and ciphertext sizes, has been extensively documented by the OQS project. Since these metrics are publicly available and maintained as part of ongoing benchmarking efforts, we refer to their official repository for the latest values [29]. Therefore, Table 1 summarizes the storage costs of the Kyber (KEM), Dilithium, Falcon, and SPHINCS+ algorithms based on their specifications in the OQS benchmarking reports. This data is particularly relevant for evaluating the feasibility of these algorithms in resource-constrained IoT environments.

On the other hand, the security levels assigned to PQC algorithms are based on NIST's classification system, which benchmarks their resistance against attacks on both classical and quantum computers. These levels help to measure the strength of a cryptographic scheme and determine its suitability for various security applications. For example, level 1 security corresponds to classical security equivalent to AES-128 or the difficulty of breaking an RSA-2048 key. Level 3 aligns with AES-192 or RSA-3072, while Level 5 is the highest, offering security comparable to AES-256 or RSA-7680. Therefore, level 5 is ideal for long-term confidentiality and high-assurance applications. However, higher security levels often come with increased computational and storage costs, requiring a trade-off between security and performance, particularly for resource-constrained devices like IoT devices. Table 1 provides a standardized way to compare the resilience of Falcon, Dilithium, SPHINCS+, and Kyber algorithms when applied in real-world cryptographic protocols such as TLS.

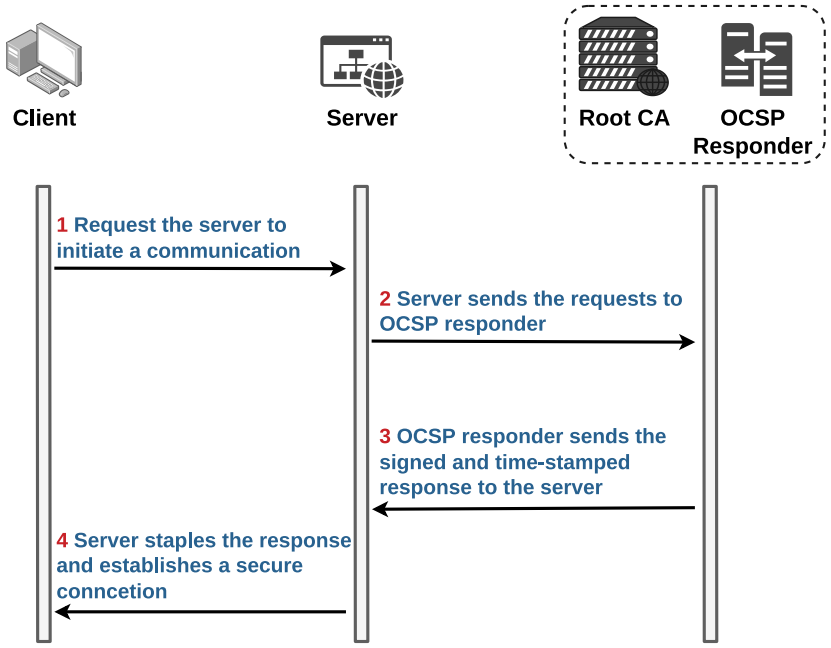

Fig. 4. OCSP stapling.

3.4. Certificate validation methods

In the context of secure Internet communications (i.e., HTTPS connections), certificate validation methods play an essential role in verifying the legitimacy and currency of digital certificates. These certificates, issued by CAs, are utilized to identify entities and secure data transmission. In case of some incidents such as private key compromise (i.e., compromise of the private key corresponding to a public key in a digital certificate or the private key that was used to sign a digital certificate), the relevant digital certificate must be revoked, and this certificate should never be used. Therefore, several methods have been proposed to check the validity of a certificate. The most commonly used certificate status check methods are certificate revocation lists and online certificate status protocol, which are explained in the following subsections.

3.4.1. Certificate revocation list (CRL)

A CRL is one of the widely used methods in the public key infrastructure (PKI) to verify the status of digital certificates and check whether the certificates have been revoked before their expiration date. A client is able to verify a certificate's serial number with the CRL in order to confirm the validity of a certificate. Thus, the client should not trust the certificate for secure communication if the CRL contains the serial number, which indicates that the certificate has been revoked.

3.4.2. Online certificate status protocol (OCSP)

Another method to determine whether a digital certificate is revoked or not is by utilizing the OCSP mechanism. Digital certificates, which are frequently used in the context of TLS along with application layer protocols such as HTTPS, are an essential part of secure communications on the Internet. A certificate can be checked using OCSP without downloading and verifying a CRL, which might lead to increasing overhead delay and the use of storage resources. Instead, OCSP enables a client to submit a request to the CA's OCSP responder and receive a response with information on the state of the concerned certificate [30]. The response could be good, revoked, or unknown in case the OCSP responder could not offer a conclusive response. On the other hand, as shown in Fig. 4, OCSP Stapling could be used to decrease the overhead delay and have more efficient communication. One advantage of OCSP Stapling over CRL is that a revoked certificate's status can be sent along with the server certificate without waiting until the next CRL update time. Moreover, OCSP stapling allows the servers to deliver the OCSP answer directly to clients during the TLS handshake. It overcomes some of the privacy, performance, and reliability issues related to the OCSP [31].

3.5. IP over BLE

IP over BLE is a technology that enables devices that feature BLE to communicate over the Internet or local network using IP-based protocols. Therefore, it allows BLE devices to transmit data packets over the Internet, extending their range beyond the typical short-range limitations of Bluetooth. The TCP/IP and BLE protocol stacks are comprised of two different sets of protocols that meet different requirements. Hence, there is no established standard for IPv4 over BLE. Nevertheless, there have been efforts

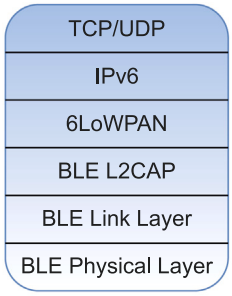

Fig. 5. IPv6 over BLE stack.

to create such a standard, such as RFC7668 [9], although no official standard has been approved yet. Fig. 5 depicts the proposed protocol stack by this RFC. It is worth mentioning that 6LowPAN is used to make IPv6 packet headers smaller.

3.6. Establishing IPv4 over BLE connections

Building a TLS connection over a BLE link is not a very common case because it is typically used to pair physically close devices. Also, it is important to note that traditional TLS implementations might not be directly applicable to BLE due to its resource-constrained architecture. Hence, the first problem to solve was to build an IP network over BLE, thus, the connection can be secured using TLS, and any BLE device with a public IP address can be accessed all over the world. Moreover, this approach requires an intermediary (e.g., a gateway device) that dispatches BLE packets coming through BLE connections to itself, to their destinations on the Internet.

We leverage some tools available on Linux and implement a simple method of running IPv4 over BLE. One of the tools that was used to set up a Bluetooth Personal Area Network (PAN) on RPI is `Bluez` tools. The Bluez Bluetooth stack in Linux has a large set of APIs to monitor and manipulate the traffic passing through the BLE interfaces. This tool provides command-line utilities for Bluetooth management, which allows the configuration of the network bridge by creating a new configuration file that defines a virtual network interface (e.g., `pan0`) and assigns it the role of a bridge. The Dynamic Host Configuration Protocol (DHCP) can be enabled directly in the network configuration file by setting up a system-network service and enabling the built-in DHCP server by simply doing "`DHCPServer=yes`," which allows the IoT devices to assign IP addresses manually or automatically to connected devices. Another feature that could be utilized is the Bluetooth authentication service, which allows the Raspberry Pi to handle Bluetooth connections without requiring manual input or output. This service will automatically start and manage authentication for connected devices.

Therefore, a TLS connection could be initiated based on this IP-based communication, allowing IoT devices to communicate with the gateway device and exchange data through the established connection. To make this TLS connection quantum-resistant we use one of the open-source implementations, i.e., `liboqs` developed by Open Quantum Safe project. For the performance evaluation of the TLS handshake protocol, we use open-source OpenSSL tool along with quantum-resistant algorithms so that we can stop client device before sending any application data.

We have two combinations of digital signature-key exchange method pairs. In the first one, PQ signature schemes are used along with ECDH key exchange whereas PQ signature schemes are used with PQ KEM schemes in the second one. The former provides quantum resistance in digital signature operations only, whereas the latter provides that in both authentication and key exchange operations.

4. Deployment and threat model

4.1. Deployment PQ model to IoT devices

The deployment of PQ-TLS in IoT devices requires consideration of cryptographic selection, integration with existing security frameworks, and mitigation of resource constraints. A practical deployment model involves adopting PQC algorithms such as Kyber for key exchange and Falcon for authentication while maintaining hybrid cryptographic support to ensure backward compatibility with legacy IoT devices that still rely on RSA or ECC. This hybrid model allows a gradual transition to quantum-safe communication while maintaining interoperability with existing IoT networks. One of the key challenges in deploying PQ-TLS is the increased computational overhead, larger key sizes, and higher bandwidth requirements associated with PQC algorithms, which require optimized cryptographic libraries and hardware acceleration techniques. Moreover, network latency and bandwidth limitations must be addressed by implementing efficient session resumption mechanisms and key caching strategies to minimize repeated PQC computations. Another critical aspect of PQ-TLS deployment is secure certificate management, as post-quantum certificates introduce larger signature sizes, impacting certificate validation efficiency. To mitigate authentication risks, IoT devices should enforce strict PQC certificate validation mechanisms through CRL and OCSP stapling, ensuring that expired or compromised certificates are not

trusted. The transition to PQ-TLS also faces regulatory challenges. IoT manufacturers must comply with emerging standards such as NIST's PQC guidelines IR 8547 [32], GDPR data protection mandates [33], and ISO/IEC 27001 cybersecurity requirements [34]. Government and industry regulations must provide clear guidelines for PQC adoption to ensure a standardized and secure transition.

4.2. Threat model

The security of the proposed PQC implementations for IoT devices using TLS communication depends on their robust and secure deployment. Therefore, we consider various threats while designing the PQC implementation and defining its security objectives. In our threat model, we assume that communication between IoT devices and the server occurs over an untrusted network, making it vulnerable to both passive and active eavesdropping. Thus, we assumed the possibility of the following types of attacks on our implementation.

1. Attack 1 — Side-Channel Attacks: In an IoT-based infrastructure utilizing PQC over TLS, the physical accessibility of devices introduces a heightened risk of side-channel attacks. To extract sensitive cryptographic information, adversaries can exploit power consumption patterns or timing variations. An attacker could potentially compromise the PQ key exchange or authentication process by analyzing these side channels without directly intercepting the encrypted communication between IoT devices and the server.
2. Attack 2 — Man-in-the-Middle (MitM) Attacks: A MitM attack occurs when an adversary intercepts and potentially alters the communication between two parties (e.g., an IoT device and a server) without their knowledge. The attacker positions themselves between the two communicating entities and can eavesdrop, modify, or inject malicious commands.
3. Attack 3 — Certificate Spoofing & Authentication Bypass: When utilizing a TLS Handshake, certificate spoofing, and authentication bypass attacks could occur when an adversary impersonates a legitimate entity (e.g., an IoT server, cloud platform, or gateway) by forging or exploiting vulnerabilities in the digital certificate-based authentication process. These attacks enable adversaries to intercept, manipulate, or decrypt communications between IoT devices and servers.
4. Attack 4 - Classical Cryptography Downgrade: In an IoT-based infrastructure employing TLS communication, a downgrade attack might occur when an adversary forces a system to use a weaker, less secure cryptographic algorithm instead of a stronger, more modern one. In the context of PQC and TLS communication for IoT devices, a classical cryptography downgrade attack exploits vulnerabilities in hybrid cryptographic implementations, coercing IoT devices to fall back to classical cryptographic algorithms (e.g., RSA, ECC).

5. Experimental configurations for PQC in TLS

This section presents our different configurations when establishing TLS connections for consumer IoT devices using PQC. Specifically, we present the details of various methodologies utilized by post-quantum signature schemes in conjunction with ECDH key exchange and KEM schemes.

5.1. First configuration - PQ key encapsulation mechanism

In this configuration, we use a chain of PQ-signed certificates for server authentication and PQ KEM schemes for key exchange. In KEM, one party uses the entropy in the recipient's public key to generate a random secret key both in cleartext and in ciphertext. Then, it sends the ciphertext to the recipient. The recipient decrypts the ciphertext and obtains the secret key by running the same KEM algorithm with its own private key corresponding to the public key used in secret key generation. Now that both parties have the same secret key they can use it to secure their communications.

In ECDH key exchange, each party mathematically computes the secret key on an elliptic curve. In contrast, in this configuration, one party generates and encrypts the secret key using the recipient's public key.

5.2. Second configuration - ECDH key exchange

This configuration also includes using a chain of PQ-signed certificates for the authentication, but it employs ECDH for key exchange. The ECDH requires both parties to randomly generate their public–private key pairs. The public keys are exchanged between the two parties, so they can agree upon a common secret key, which will be used in symmetric key operations such as encryption/decryption and message authentication code calculation.

In this configuration, we prefer to use the Curve25519 elliptic curve instead of the NIST P-256 curve as the former consumes computational resources less than the latter [35], making it more convenient for IoT devices, which have limited processing resources [36].

5.3. Third configuration - Mutual authentication

Mutual authentication is also known as a two-way authentication or mutual SSL authentication. It is a security process in which both parties verify each other's identities in communication. This process introduces an additional communication overhead compared to a single authentication strategy. However, while mutual authentication increases the overhead, it enhances security by ensuring that both parties involved in the communication are who they claim to be and mitigating risks that are associated with impersonation or man-in-the-middle attacks.

For our tests, we create a chain of certificates that are signed using the PQ digital signature algorithms, and the root and the intermediate CAs' private keys. These PQ certificates authenticate the server and the client (when mutual authentication is enabled). The TLS handshake protocol dictates the server to present all the certificates in its chain except for the root CA's certificate to the client so that the client can validate the authenticity of the server. It is worth noting that the client has an authentic copy of the root CA's certificate that is digitally signed using the PQ algorithms.

6. Dynamic switching for PQ cryptography

Implementing PQC on constrained IoT devices introduces several challenges, such as computational complexity, communication overhead, and memory requirements. Moreover, in order to address these challenges with the limited resource devices, developing optimized implementations of PQC algorithms can focus on minimizing the computational complexity and communication overhead. That is, PQ KEM and digital signature algorithms in use should be configurable based on requirements/constraints peculiar to the device and/or the application.

To this end, we propose a new approach when establishing a TLS session for a consumer IoT device. The main motivation behind this approach is to offer dynamicity in terms of PQ algorithm selection. Depending on the available resources and interfaces, an IoT device acting as a client in a TLS session can change the TLS-supported curve configurations to offer the most optimal performance. Specifically, this approach allows us to dynamically choose the underlying TLS signature and KEM algorithms based on the network interface in use, such as Wi-Fi or BLE, as their data rates are much different. Higher data rates may accommodate larger signatures better. Moreover, an IoT device may have multiple communication interfaces, and the device may be able to select either one of them based on data rate requirements of an application. Such a device may want to select higher security level algorithms when a communication interface with high data rate is attached if the energy consumed is not a big concern (e.g., if the device operates on power supply). Furthermore, latency constraints of the application may be considered at selecting suitable PQC algorithms. These are some example metrics (i.e., interface device type, power source, and latency constraints) that can be used by the device to select an appropriate PQC algorithm pair. This can be executed as a service after all relevant metrics are determined.

To offer this service, we retain the interface when a TLS connection is initiated with an outside server. In addition, we retrieve the power source on which the IoT device operates (e.g., power supply or battery), and we assume that the latency constraints of an application are downloaded when it is installed. We then check this information to change the supported groups and signature algorithm in the extension section of the TLS handshake message accordingly. This way, the server will respond with the requested cipher suite, and thus, the rest of the TLS handshake utilizes the selected PQ algorithms.

Fig. 6 shows a rough overview of the dynamic PQ suite selection process when establishing a TLS connection. Whenever an application running on the client needs a TLS connection to a service running on the server, the application drills down the layered architecture of the network stack and sends probes to find out what type of physical network interface (i.e., BLE or Wi-Fi) on the device is bound to the route towards the server. Once the interface is confirmed, the application picks the appropriate PQ KEM and digital signature algorithms and modifies the supported groups and signature algorithms extensions of the *ClientHello* Message in the TLS handshake protocol as shown in Fig. 7.

Technically speaking, this will require a cross-layer approach. However, we propose to extract this information without the need to change the protocol stack. Specifically, the application first resolves the server's domain name (or uses its IP address if already known) and initiates a TCP connection. Once the connection is established, the application queries the route to the target IP address. *ip* utility's *route* object [37] can be used for that purpose. The output of this command includes not only the network interface but also some other supplementary information, such as source and destination IP addresses. Therefore, the output is piped to another command in order to *grep* the network interface. However, the interface name is a logical name to represent the physical device and does not specify the device type, e.g., Ethernet, Wi-Fi, Bluetooth, etc. *nmcli* [38] tool can be used to resolve the device type.

Now that we have obtained the device type, we can select the appropriate PQ suite and modify the *ClientHello* Message accordingly. Once the KEM and digital signature algorithms to be used are determined, we inject them into supported groups and signature algorithms extensions of the message, respectively.

7. Security analysis

In this section, we provide an assumption analysis-based security analysis of the proposed protocol against the threat model defined in Section 4.2.

Integrating PQC in IoT devices utilizing TLS communication introduces several security challenges, particularly regarding MitM and certificate spoofing attacks. These threats target different aspects of implementation, exploiting vulnerabilities in key exchange, authentication mechanisms, and cryptographic processing. A robust security model must be implemented to reduce these threats and ensure secure communication between IoT devices and servers.

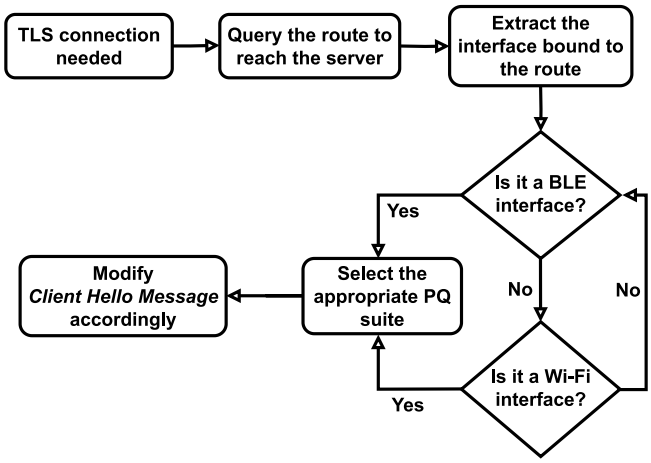

Fig. 6. Dynamic TLS with PQC.

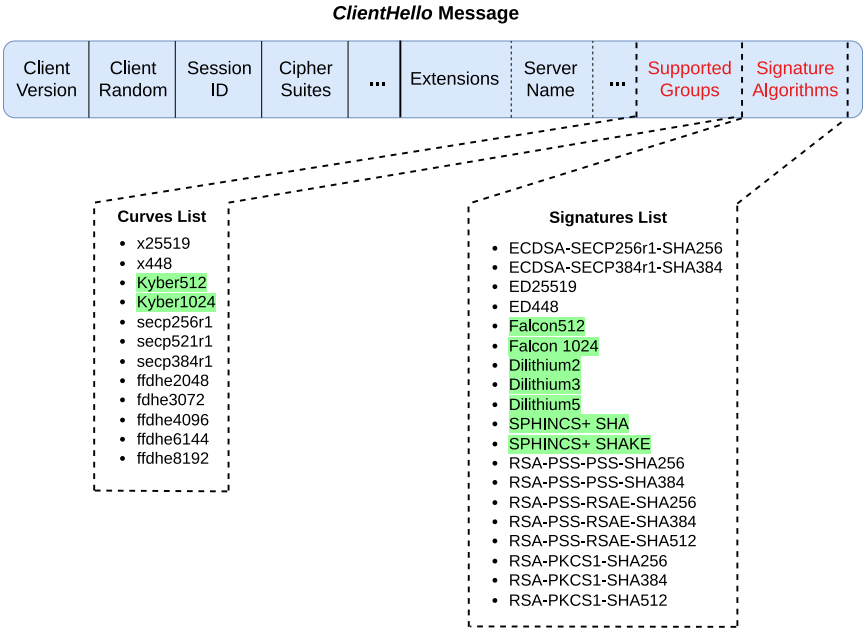

Fig. 7. *ClientHello* message.

Side-channel attacks (Attack-1) represent a category of security vulnerabilities that derive information from the practical execution of a computing system rather than weaknesses in the algorithms that have been implemented. These attacks exploit physical implementations that leak information, which can be exploited to retrieve confidential data, including cryptographic keys. One of the common aspects of side-channel attacks is the examination of the patterns and statistical features that may arise from repeated use of the same cryptographic keys. Therefore, it poses a significant risk to IoT devices due to their reliance on power-efficient hardware, which may lack protection against physical and remote analysis techniques. Attackers could also introduce faults in cryptographic computations through power glitches or electromagnetic pulses, leading to exploitable errors in PQC key generation. Implementing constant-time cryptographic operations [39] on IoT devices will ensure that execution time and power consumption remain the same regardless of input data. Moreover, masked implementation can be easily enabled using the OpenSSL Library to introduce randomness into computations, preventing attackers from deriving important information through side-channel analysis [40,41].

MitM attacks (Attack-2) in PQC-IoT-TLS are particularly dangerous as they allow attackers to intercept and manipulate key exchange messages and compromise the confidentiality and integrity of communications. Since many PQC key encapsulation mechanisms, such as Kyber, involve public-key exchanges, an adversary positioned between an IoT device and the server can alter key exchange messages, either forcing weaker cryptographic parameters or injecting maliciously crafted keys. Therefore,

implementing mutual authentication between the IoT devices and servers using PQC-based digital signatures, such as Dilithium or Falcon, along with PQC-compliant digital certificates, could be critical in defending against MitM attacks. Digital certificates issued by a trusted CA bind the public key to the device or server's identity, preventing attackers from using corrupt keys. These authentication mechanisms ensure that both parties verify each other's identities before establishing a secure connection, preventing adversaries from injecting false credentials.

Certificate spoofing and authentication bypass attacks (Attack-3) further compromise the security of PQC communication by allowing adversaries to manipulate digital certificates, tricking devices into trusting malicious entities. They can impersonate a trusted server or device, bypassing authentication and gaining unauthorized access. In order to defend against certificate spoofing, IoT devices must enforce strict validation mechanisms of post-quantum certificates, ensuring that only those signed by trusted post-quantum certificate authorities are accepted. Therefore, OCSP stapling should be used to verify certificate revocation status, preventing devices from trusting expired or compromised certificates.

Classical cryptography downgrade attacks (Attack-4) remain one of the most insidious threats to PQC implementations, as they enable adversaries to weaken security by forcing the IoT devices into falling back to classical cryptographic algorithms that are vulnerable to quantum decryption. Thus, once downgraded, communication remains vulnerable to future quantum attacks, allowing adversaries to decrypt stored encrypted data once quantum computers become powerful enough. Therefore, to mitigate downgrade attacks, IoT devices should enforce PQC-only security policies, ensuring that classical cryptographic fallback is not permitted once PQC algorithms are supported. While PQC can be integrated with the old versions of TLS 1.1 and 1.2 in hybrid modes, both versions have their weaknesses and are prone to various cryptographic attacks. Hence, TLS 1.3 should be strictly enforced to prevent potential downgrade attacks, and any attempt to negotiate weaker cryptographic parameters should result in connection termination.

8. Testbed implementation

To evaluate the performance of TLS handshake using a full PQ communication network, we set up a testbed and created a similar IoT environment to the client–server model in which the client (e.g., PC/Laptop/IoT Device) communicates with a server (e.g., Web Server) through an IoT gateway via Wi-Fi or BLE. As shown in Fig. 8, the Raspberry Pis (RPIs) simulate an IoT device as a client as well as the IoT Gateway. The laptop used in the setup acts as a server, and the gateway connects the IoT device to the server. Moreover, the CAs and the CRL/OCSP servers are hosted on the Google Cloud Platform (GCP).

The Hardware and Software: We used two RPI 4s for the client and IoT Gateway; thus, we used a laptop as a server. The RPIs have 4 GB RAM and a 1.8 GHz ARM processor, and support BLE features. The laptop has 8 GB RAM and a 3.2 GHz Intel i7 9th generation processor, runs Ubuntu 20.04, and also supports BLE features. All the materials to reproduce the same experimental environment, including PQ certificates that were used in the experiments as well as OCSP/CRL servers, are available on GitHub[1].

Extraneous Variables: To control extraneous variables within the testbed environment, we implemented the following measures:

1. Controlled Distance and Environment: The client and server were placed 30 meters apart in a laboratory setting to maintain a controlled physical environment.
2. Network Congestion Management:
    - For Wi-Fi experiments, a private network (e.g., a personal hotspot) was used instead of a congested public Wi-Fi to avoid any interference.
    - For BLE experiments, a private bridge network was integrated into the gateway between the server and the client to ensure minimal external influence.
3. Consistency in Hardware and Software: The same hardware and software configurations were maintained for all test runs to eliminate variations due to system differences.
4. Multiple Iterations and Result Averaging: Each experiment was run multiple times, and the average results were taken to mitigate random performance fluctuations.

By implementing these controls, we ensured that the results were as reliable and repeatable as possible while minimizing the interference of external factors.

To employ PQ algorithms in the TLS handshake protocol between two parties, the initial step involves acquiring, building, and installing `liboqs` (version 0.11.0) on both devices. On the laptop, one can proceed with installation by following the instructions specified in the wiki page of the relevant GitHub repository [26]. However, cross-compilation is needed for the Raspberry Pi due to its ARM processor.

IP Connections: Initially, the IoT Gateway was set up as an access point, and its interface was bound to the BLE radio. The open-source package named `bluez-tools` (version 5.73) [42] was used to configure the radio. Lastly, we paired the gateway with the client through the BLE link to create a BLE network.

That being said, the client acquired an IP address from the network via the IoT Gateway, employing this address for identification and data exchange within the network. Subsequently, `OpenSSL` (version 1.1.1) was utilized to establish a TLS connection between

[1] https://github.com/adwise-fiu/PQ-Cryptography-With-Certificate-Validation-Methods

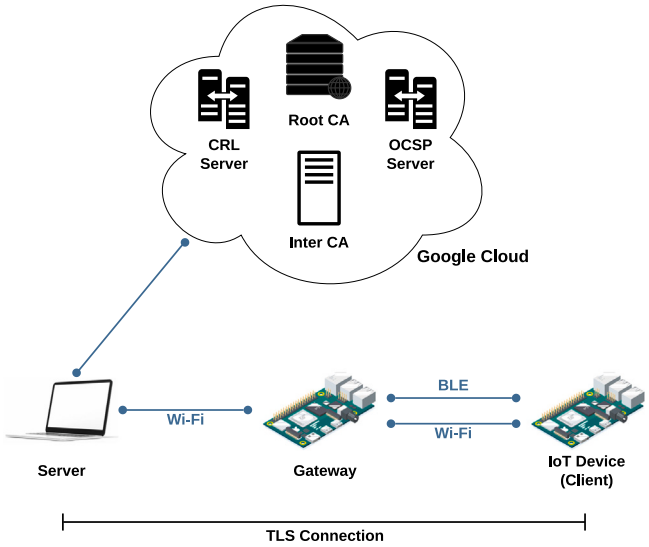

Fig. 8. Testbed setup.

the client (RPI) and the server (laptop) within the BLE network. The BLE communication that has been used in the setup can be considered as hybrid communication in which the communication between the client and the IoT Gateway is through IP over the BLE interface, and then the IoT Gateway forwards the data via a bridge to the Wi-Fi interface towards the server. This hybrid communication represents the same features as Minew's G1 IoT gateway [10].

We enabled PQ CRL and OCSP servers on the cloud. The CRL and the OCSP servers were accessed by the server that interacts with the client depending upon the experiments being carried out, i.e., those in which the server needs to send the status of the X.509 digital certificates.

For the CRL experiment, serial numbers of revoked certificates that a CA generated are stored in a CRL and maintained on the CRL server. Therefore, the CRL had it signed by the Root CA, resulting in a file size of approximately 10 MB. To increase the CRL size, we attempted to merge another CRL, but this was not feasible due to the trust model enforced for the CRLs. Since each CRL is issued and signed by a specific CA, merging CRLs from different CAs would break the integrity and authenticity of the revocation list, making it untrustworthy. Additionally, maintaining a relatively small CRL size would be beneficial for our use case, as we are working with IoT devices, which are inherently constrained in terms of memory and storage. Ensuring a lightweight CRL is critical to optimizing performance and ensuring that revocation checking remains efficient in resource-limited environments.

for the OCSP Stapling experiment, the CA that created a certificate for the server maintains an OCSP server. Thus, the server requests the OCSP server to inquire about the certificate's status. After that, the OCSP server sends the signed and time-stamped response to the server; the response will be stapled and forwarded to the client, thereby not needing to download any file for the validity check of the server certificate. On the other hand, enabling caching on the server side will avoid this latency, and the response can be cached for a certain period of time (i.e., up to 7 days [43]). The size of OCSP caching depends on several factors, including the number of certificates being validated and the storage mechanism used by the client or server. Therefore, each cached response for more than 45,000 users is about 100 MB, so each response is typically about 2 KB in size. When an OCSP response is cached, it includes specific headers that indicate its expiration time, such as `cache-control, age, and X-cache` which indicates that the response was served from cache or taken from the origin server.

A chain of certificates consisting of a root certificate authority (CA) and an intermediate CA has been used to sign an end entity (server) certificate to observe its communication and computational overhead in a realistic setup. Also, the intermediate CA signed the client certificate in order to evaluate the mutual authentication experiments. Thus, we generated a trusted self-signed root CA certificate using the OpenSSL PQ library, and for each PQ signature scheme, four certificates are associated with each chain of certificates. The Wi-Fi approach was executed by utilizing the IoT Gateway's built-in "Wi-Fi Hotspot" feature, establishing a direct wireless link between the client and the server to ensure that the Wi-Fi and BLE was compared fairly. For this procedure, we followed the same steps mentioned before as we did for gathering data.

During dynamic switching experiments, the `ip` utility's (version 6.1.0) *route* object and *get* switch have been utilized to retrieve the network interface used to access the IP address given as an argument. Another network management tool called `nmcli` (version 1.46.0) has been used to resolve the device type. Detailed information about the network interface can be retrieved by using the *nmcli* tool along with *device* and *show* switches and providing the logical interface name obtained from the *ip* utility. The output of this command is highly structured. Specifically, the device type was easily extracted by *grep*ping with **`GENERAL.TYPE`** keyword and discarding the first block of the output. To manipulate the *ClientHello* message, there are two ways: Command line options and **`SSL_CONF_cmd`** function (**`SSL_CONF_cmd_value_type`** as of OpenSSL 3.0). If the application is a bash script, OpenSSL's

Algorithm 1 Dynamic switching algorithm

```
Require: Domain name of target server: domainName
ipAddress ← DNS_Resolution(domainName)
tcpConnection ← TCP_Connect(ipAddress)
networkInterface ← IP_Route_Get_Interface(ipAddress)
generalProperties ← NMCLI_Show_Device(networkInterface)
for each property ∈ generalProperties do
    if property.name = "type" then
        deviceType ← property.value
        break
    end if
end for
KEMAlg, SigAlg ← Retrieve_Suitable_PQC_Suite(deviceType)
clientHello ← SSL_Client_Hello()
SSL_CONF_cmd(clientHello, "groups", KEMAlg)
SSL_CONF_cmd(clientHello, "sigalgs", SigAlg)
Initiate_TLS_Handshake(clientHello)
```

command line tool can be used to manipulate *groups* and *sigalgs* options. Similarly, `SSL_CONF_cmd` function can be used to configure supported groups and signature algorithms by providing the same options and lists of supported algorithms in a colon-separated format. Steps to be followed for dynamic switching are given in Algorithm 1.

Measuring the Connection Speed: The fundamental OpenSSL library contains built-in commands that are helpful for our needs. To validate that the connection is working properly and utilizing the required algorithms, we followed these steps:

- The first step was to use OpenSSL to generate a certificate chain for all of the signature algorithms, which would be used for testing in both this section and the next.
- The next step was to enable the certificate validation methods and use the OpenSSL commands `s_server` and `s_client` to establish a connection over the hybrid network and initiate the TLS handshake protocol.
- The final step was obtaining the average communication delay. We calculated averages of the wall-clock time recorded for each run and got the number of connections made during that specific run, in order to obtain the average connection delay for each PQ signature scheme using ECDH key exchange.

To measure the average communication delay of each algorithm, we used `TCPDUMP` tool (version 4.99.5) [44], which is used to capture and analyze network packets transmitted by the RPI and the laptop. The network interface can be applied to capture relevant packets and apply specific ports, IP addresses, or entire network ranges. Moreover, to optimize performance, users can limit the number of captured packets, adjust buffer sizes to prevent drops, and capture only the necessary portion of each packet. The captured data can be saved as a `pcap` file and later can be analyzed using Wireshark [45]. This recording served to calculate the total time for each run as well as to decide the most suitable PQ algorithm for a BLE network. Regardless of the operating system, this configuration can be reproduced on any machine as long as it can run OpenSSL.

9. Performance assessment

In this section, we present the evaluation metrics and results of the experiments involving PQ digital signature and KEM schemes. We note that the results were repeated 30 times and are within (mean ±1.96 * standard error) with a 95% probability.

9.1. Performance metrics and baseline algorithms

We performed several tests to measure how long each of the quantum-resistant KEM and digital signature algorithms is prevented from establishing a TLS connection. Our primary metric is network latency when a client and a server establish a TLS connection before data transmission begins.

We recorded messages containing digital certificates and signatures generated using classical standards such as EdDSA, ECDSA, and RSA as a baseline for the PQ digital signature algorithms. In addition, the Wi-Fi was used as a benchmark for comparison to the BLE. To assess the impact of BLE's lower bandwidth on algorithm performance, we also acquired data on a Wi-Fi connection over the same distance as a baseline for comparison with our BLE experiment results. Moreover, since the OCSP server is hosted on the GCP, the OCSP Stapling response of each signature scheme is cached on the server side.

We also tested the effect of some certificate validation methods on the latency of TLS connection establishment. As a baseline, we used a scenario where no certificate validation method is employed. In the baseline, it is assumed that the certificate provided by the server is neither revoked nor expired.

Table 2
Comparison of the handshake delay for ECDH with various signatures and certificate validation methods (ms).

	OCSP stapling		CRL		No Cert. Vali	
Signature scheme	**BLE**	**Wi-Fi**	**BLE**	**Wi-Fi**	**BLE**	**Wi-Fi**
EdDSA	189.9	76.5	127.1	48.84	122.4	41.9
ECDSA	204.9	98.0	151.7	60.81	146.7	56.7
RSA	221.3	108.4	190.6	80.42	178.3	64.0
Falcon512	254.8	127.3	207.3	102.8	196.2	101.5
Falcon1024	291.5	150.7	219.6	125.22	214.7	118.8
Dilithium2	356.2	312.4	268.71	162.5	223.5	152.2
Dilithium3	403.7	329.5	278.4	209.4	263.6	191.9
Dilithium5	465.3	376.1	332.1	224.7	316.4	198.7
SPHINCS+ SHA	822.2	541.5	445.7	302.65	424.9	275.7
SPHINCS+ SHAKE	1006	722.3	668.3	416.42	653.7	352.7

9.2. Evaluation of PQ signature schemes and ECDH key exchange with different certificate validation methods

In this section, we present and discuss the results obtained from the experiments conducted using the combination of ECDH key exchange and PQ digital signature schemes along with enabling different certificate validation methods such as OCSP stapling and CRLs. Our CRLs contain serial numbers of 10 revoked certificates that our CA has signed. It offers an insight into the way the PQ digital signature schemes perform on their own without the use of PQ KEM schemes. Following the decisions made by NIST [46], we removed SPHINCS+ Haraka and the robust version from the tweakable hash function.

Table 2 lists the experiment results for the test cases where OCSP stapling, CRL, and no certificate validation methods are employed. The tests were carried out separately for the hybrid BLE and pure Wi-Fi connection. The results in the table show that OCSP stapling notably increases handshake delay. The Falcon-512 signature scheme outperforms all the other selected PQ signature schemes over a BLE connection although it is slightly slower than the baseline schemes. Dilithium2 without OCSP mechanism performs similarly to the benchmark outcomes when used via Wi-Fi. Nevertheless, due to the fact that the size of digital signatures generated by the Dilithium family of signature schemes are larger compared to Falcon [47], it is possible that Dilithium2's performance suffered over BLE. Larger signatures causes some additional delay when sending the certificate in a limited bandwidth environment such as BLE. The SPHINCS+'s enormous signature sizes brought on longer delays, which resulted in the worst performance across both BLE and Wi-Fi.

The two best PQ digital signature algorithms, Falcon and Dilithium showed an interesting pattern of behavior when OCSP is enabled such that the overheads differ significantly among them. For the Falcon family, both BLE and Wi-Fi based TLS increase for a similar amount (e.g., on average, BLE latency increases by around 30% and Wi-Fi latency increases by 25%). However, for the Dilithium family, the increase is much more significant for Wi-Fi. As an example, for Wi-Fi, the increase is around 90% on average, while for BLE, this is around 55% on average. This tells us that as BLE bandwidth is already limited and packets are probably fragmented, the addition of OCSP does not make a great impact on an already slow handshake. However, this makes a big difference for Wi-Fi as OCSP round trip times are more significant compared to client-to-server propagation delays. The outcome is that Falcon family performance is much more stable under changing configurations.

The results also clearly show that checking the validity of a certificate against a CRL is faster when compared to OCSP stapling. As an example, for Falcon512, the increase in delay with respect to no certificate validation is 23% for BLE whereas it is 20.2% for Wi-Fi. Depending on the application requirements for data latency, this may be a factor to consider CRL approach. Nonetheless, given the limited resources of some IoT devices (e.g., fitbit), using the OCSP mechanism will be more storage efficient than CRL.

9.3. Evaluation of cached OCSP stapling

This experiment focuses on optimizing OCSP, given its advantage in terms of storage. We evaluate how much caching capability can improve the performance when caching is done by the TLS server. Specifically, it obtains the OCSP response from the OCSP server for every TLS handshake in the Non-Cached OCSP stapling and will not store the response in the cache; this introduces additional latency and increases the load on the OCSP server. On the other hand, enabling the caching on the server side will avoid this latency, and the response can be cached for a certain period of time (i.e., up to 7 days [43]). Thus, we conducted some experiments to determine the OCSP response time in this instance, such as Cached and Non-cached responses on the server side.

The experiment was based on a realistic environment in which both servers were hosted on the Google Cloud Provider (GCP). While the OCSP server was hosted on the West Coast, the TLS server was on the East Coast. Thus, when the OCSP responses are stapled, they can be cached by the server, which means that the subsequent requests for the same certificate from clients can be answered immediately with the cached OCSP response. This caching mechanism significantly reduces the time required to validate the certificate's revocation status. For instance, as shown in Fig. 9, the caching OCSP response for Falcon512 has approximately a 42.3% latency decrease compared to a Non-cached OCSP response. These results indicate that cached solutions would be attractive for IoT devices in the era of PQ.

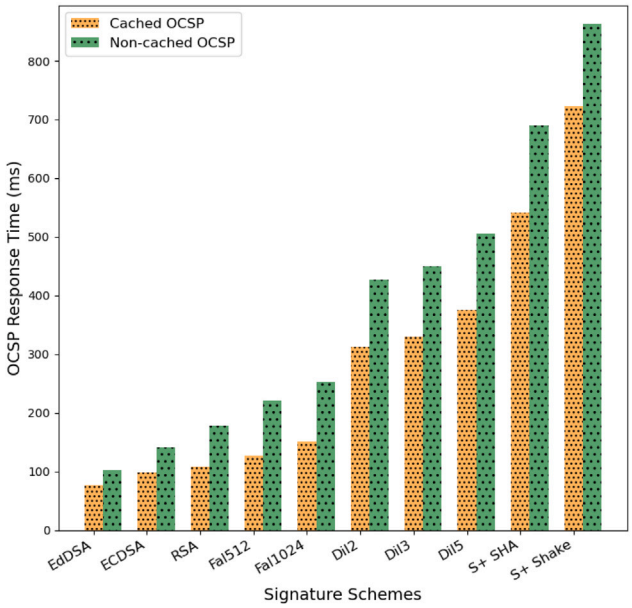

Fig. 9. Result of Non/Cached OCSP stapling response.

9.4. Evaluation of PQ KEM and digital signature schemes with different certificate validation methods

In our following experiment, we evaluated the performance of the PQ digital signature and KEMs together. To this end, a PQ KEM was employed for key exchange, and the digital certificate transmitted in the TLS handshake protocol was signed by a PQ digital signature algorithm. The average delays for connection establishment over the Wi-Fi and BLE when PQ digital signatures and KEM schemes are combined are shown in Table 3.

Wi-Fi and Bluetooth are abbreviated as WF and BLE, respectively. Also, for layout purposes, SPHINCS+ is denoted as S+. The average delays reported in this section were computed using the same method described in Section 9.2 above.

Referring to the figures in Table 3, all combinations performed around 12% worse than utilizing ECDH key exchange from Table 2 since the PQ key management scheme's increased complexity. When the hybrid connection BLE is used, the gap is wider. We can observe that there is a 14.71% increase for Dilithium2 and 18.72% for Falcon512, which indicates that using Kyber512 KEM method could be better for IoT devices running protocol stacks similar to BLE.

From the signatures' point of view, Falcon-512 outperformed all of the other signature algorithms. Falcon-1024 and Dilithium2 followed Falcon-512 in the evaluations. The combination of Falcon-512 and KEM algorithms, such as Kyber512, has the fastest overall performance via Wi-Fi and BLE. The least effective combinations, such as SPHINCS+, typically had more than a second over BLE connection delays and more than seven-tenths of a second over Wi-Fi. The SPHINCS+ family exhibits rather significant latency due to its extremely large signature sizes [47], which cause lag in the BLE network's constrained bandwidth environment and to a lesser extent in the Wi-Fi connections, which have wider bandwidth in contrast with BLE channels.

From the certificate validation methods' point of view, OCSP stapling and CRLs show a similar tendency as in Table 2. When latency increase rates are examined with respect to the no certificate validation method, it can be seen that the increase rates by the CRL method range from 1.06% to 19.89%, whereas those by the OCSP stapling vary between 12.45% and 138.46% irrespectively of the KEM and digital signature algorithms and network interface types.

The PQ digital signature algorithms considered in this paper claim different security levels based on the NIST's metrics. Among the level 1 signatures, the Falcon scheme requires less time and is more consistent than SPHINCS+, regardless of the certificate validation method, network interface type, and the KEM algorithm. Dilithium2 and Dilithium3 are the only level 2 and level 3 signature algorithms, respectively. As expected, Dilithium2 takes less time for any case. It is worth noting that the OCSP stapling significantly increases the time required for the TLS handshake. Falcon1024 and Dilithium5 are the only level 5 signature algorithms. The most remarkable observation here is that the delay increase rate for Dilithium5 with Kyber1024 on Wi-Fi when the CRL method is used is only 0.95% whereas that for Falcon1024 is 8.70%.

As the security level increases, the time to establish a TLS connection absolutely increases. However, as we discussed above, the increase rates may be affected differently depending on the certificate validation method or the KEM algorithm used.

To summarize, BLE connections took much longer than Wi-Fi connections. Besides, it is worth noting that Falcon family schemes show comparable performance to RSA and ECDSA for both Wi-Fi and BLE, which makes them promising options for IoT applications.

Table 3
Comparison of the handshake delay for various signature and KEM schemes (ms).

Signature scheme	OCSP stapling		CRL		No Cert. validation	
	KEM algorithm					
	Kyber 512	Kyber 1024	Kyber 512	Kyber 1024	Kyber 512	Kyber 1024
EdDSA WF	106.4	133.4	79.45	89.3	66.4	81.7
EdDSA BLE	211.5	258.4	170.2	208.4	167.4	198.6
ECDSA WF	117.3	192.6	87.52	100.8	73.0	94.3
ECDSA BLE	233.4	282.2	194.9	221.6	185.6	210.3
RSA WF	126.7	211.1	95.64	119.6	86.7	114.1
RSA BLE	273.2	332.5	237.8	274.6	222.5	268.9
Falcon512 WF	148.6	204.7	133.01	171.18	122.6	158.4
Falcon512 BLE	302.5	397.1	254.1	320.8	247.3	313.7
Falcon1024 WF	191.1	237.5	155.89	189.69	140.9	174.5
Falcon1024 BLE	372.2	434.3	320.4	393.1	301.6	384.1
Dilithium2 WF	362.3	419.4	175.1	207.6	160.5	195.0
Dilithium2 BLE	408.6	515.8	332.9	447.1	327.9	439.3
Dilithium3 WF	383.5	487.7	216.1	248.2	201.4	240.8
Dilithium3 BLE	482.2	571.8	407.6	517.7	395.7	508.5
Dilithium5 WF	402.8	562.4	270.3	319.4	252.0	316.4
Dilithium5 BLE	558.9	613.4	433.2	557.7	418.5	542.2
S+ SHA WF	751.4	1063	350.01	541.3	315.1	525.3
S+ SHA BLE	1143	1343	747.2	824.4	733.6	806.9
S+ SHAKE WF	983.3	1237	560.88	652.66	533.8	624.2
S+ SHAKE BLE	1361	1568	754.1	882.5	746.2	862.5

9.5. Evaluation of mutual authentication

We conducted some experiments to determine the overhead in this instance, such as mutual authentication, compared to the single authentication strategy because it is a common practice for servers in an IoT network to ensure that the data they receive is from a legitimate source. Since it appeared that mutual authentication would result in an increase in overhead, we only chose the two top options from the prior results, i.e., Dilithium2 and Falcon-512. Table 4 illustrates this trend using both BLE and Wi-Fi network connections. Therefore, compared to Table 3, all instances of mutual authentication showed an increase in TLS handshake delay when compared to that of single authentication, which is expected given the additional certificate transmission, validation, and verification needed.

Over Wi-Fi, a combination of Kyber512 and Falcon-512 took 148.6 ms and 216.4 ms in average for single and mutual authentication, respectively, which means there is an increase in latency by 45.6%. Over BLE, we observed some interesting results. It took 302.5 ms and 354.98 ms for single and mutual authentication, respectively, where the increase is only 17.3%. We speculate that this is a similar situation we discussed in Section 9.2. As the latency for BLE is already high, additional overhead does not sharply increase the overall latency for TLS handshake, which is not the case for Wi-Fi. This means BLE-based mutual authentication will be more stable in performance compared to Wi-Fi.

9.6. Evaluation of hybrid KEM and digital signature schemes

We evaluate hybrid PQC-classical configurations using TLS handshake delay measurements, presented in Table 5. The table compares the handshake delays for various hybrid digital signature schemes (e.g., P256/Falcon512, P256/Dilithium2, and P256/Sphincs+SHA) with different KEM such as X25519, P256/Kyber512, and P521/Kyber1024 over Wi-Fi (WF) and Bluetooth Low Energy (BLE) connections. The Table lists the experiment results for the test cases where OCSP stapling and no certificate validation methods are employed. The results in the table show that OCSP stapling introduces a measurable overhead, increasing handshake delay compared to the no certificate methods. For instance, EC(P256) and Falcon512 for Wi-Fi experience a 7.6% delay increase due to the OCSP Stapling. Furthermore, BLE connections suffer significantly higher delays than Wi-Fi, around 25.5%

Table 4
TLS delay with mutual authentication for different validation methods (ms)

Signature Scheme	OCSP Stapling		
	ECDH	Kyber512	Kyber1024
Falcon512-WF	183.9	216.4	287.2
Falcon512-BLE	297.2	354.98	431.85
Dilithium2-WF	369.9	404.9	463.7
Dilithium2-BLE	414.55	476.98	543.41
	CRL		
	ECDH	Kyber512	Kyber1024
Falcon512-WF	164.2	195.7	236.7
Falcon512-BLE	271.6	311.4	385.2
Dilithium2-WF	264.3	324.3	347.8
Dilithium2-BLE	352.3	394.1	471.3
	No Cert. Validation		
	ECDH	Kyber512	Kyber1024
Falcon512-WF	131.65	184.27	228.43
Falcon512-BLE	236.4	286.7	341.8
Dilithium2-WF	206.85	239.2	284.5
Dilithium2-BLE	269.7	363.7	460.8

Table 5
TLS delay for hybrid signatures and KEM (ms).

Hybrid Signature	OCSP Stapling		
	X25519	P256/Ky512	P521/Ky1024
P256/Fal512-WF	162.7	187.18	220.98
P256/Fal512-BLE	243.91	266.18	305.59
P256/Dil2-WF	181.81	198.85	240.82
P256/Dil2-BLE	358.92	404.79	514.79
P256/S+SHA-WF	635.43	647.14	659.35
P256/S+SHA-BLE	786.47	801.45	827.28
	No Cert. Validation		
	X25519	P256/Ky512	P521/Ky1024
P256/Fal512-WF	151.25	174.14	207.1
P256/Fal512-BLE	167.3	189.48	241.16
P256/Dil2-WF	165.97	190.15	225.67
P256/Dil2-BLE	249.61	264.42	325.33
P256/S+SHA-WF	391.8	411.14	432.06
P256/S+SHA-BLE	423.49	467.55	554.41

slower when comparing P256/S+SHA along with hybrid KEM P521 and Kyber1024. On the other hand, there is around 15% overhead when we compare ECDH(X25519) to the hybrid P256 along with Kyber512 for Falcon512/Wi-Fi OCSP. Moreover, there is a 30%–35% increase in TLS handshake when using the highest hybrid security (NIST L5 security), which is P521/Kyber1024. The most computationally expensive algorithm was P256 along with S+SHA, which P256 with Falcon512 for Wi-Fi is the most efficient hybrid PQC. These results provide a realistic perspective on PQC adoption in IoT environments, where resource constraints and communication overhead are critical considerations. The results suggest that while hybrid TLS configurations enhance security, they also introduce non-negligible delays, especially in constrained networks like BLE. As a result, optimizing hybrid cryptographic implementations such as reducing certificate validation overhead or leveraging more efficient post-quantum signature schemes—will be key to enabling secure and practical PQC deployment in IoT.

9.7. Dynamically choosing KEM/digital signature algorithms

Our findings in previous experiments using OCSP/CRL with PQ algorithms indicated that the choice of PQ algorithms may be connection and device-specific. In other words, it would be desirable to choose the underlying TLS KEM and/or digital signature algorithms dynamically based on the network interface in use (e.g., Wi-Fi, BLE, ZigBee).

In order to show the feasibility of dynamic switching, we implemented this approach through a shell script in order to execute the dynamic switching based on the interface used in the connection. The main time-consuming step in the procedure is to extract the interface information, which takes approximately 30 ms in average. This is a small computational overhead (both in terms of CPU resources and delay) because this is performed only once at the first connection towards the target server and used as is for all subsequent connections unless the data rate requirements of the application change. This means the performance of TLS handshake will not be impacted at all.

Based on the results we have obtained so far, we propose a decision making criteria on KEM/digital signature selection in Algorithm 2 for an IoT device with Wi-Fi and BLE interfaces. The algorithm depends on type of the communication device, power source of the IoT device, and latency constraints of the application that uses the connection. If the connection is established over Wi-Fi the algorithm first checks whether the IoT device operates on batteries or a power supply. If it operates on power supply

Algorithm 2 KEM/digital signature selection criteria

```
Require: Type of the device: deviceType ∈ {Wi-Fi, BLE}
Require: Power source: powerSource ∈ {power supply, battery}
Require: Latency constraints of the application: latConstraints ∈ {low, high}
if deviceType = "Wi-Fi" then
    if powerSource = "power supply" then
        if latConstraints = "low" then
            KemAlg, SigAlg ← {Kyber1024, Dilithium5}
        else
            KemAlg, SigAlg ← {Kyber1024, Falcon1024}
        end if
    else
        if latConstraints = "low" then
            KemAlg, SigAlg ← {Kyber512, Dilithium2}
        else
            KemAlg, SigAlg ← {Kyber512, Falcon512}
        end if
    end if
else
    if powerSource = "power supply" then
        if latConstraints = "low" then
            KemAlg, SigAlg ← {Kyber1024, Dilithium5}
        else
            KemAlg, SigAlg ← {Kyber512, Falcon512}
        end if
    else
        KemAlg, SigAlg ← {Kyber512, Falcon512}
    end if
end if
```

the algorithm checks latency constraints of the application assuming that data reporting profile of the application is downloaded when it is installed. If data reporting period is long, i.e., low latency constraints, *Kyber1024-Dilithium5* pair is used. Otherwise, *Kyber1024-Falcon1024* pair is used. We do not downgrade KEM algorithm to *Kyber512* because such a change saves 40 ms at most while replacing Dilithium5 with Falcon1024 saves 130 ms (Please see Table 3). If the IoT device operates on batteries the latency constraints are checked again. If the application profile shows low latency characteristics *Kyber512-Dilithium2* pair is used. KEM and digital signature security is sacrificed to reduce energy consumption. Similar to the power supply case above, we replace Dilithium2 with Falcon512 in order to meet the latency constraints. If the connection is over BLE the power source is checked. If it operates on a power supply latency constraints are checked. If data communication periods of the application is long enough *Kyber1024-Dilithium5* pair is used. Otherwise, *Kyber512-Falcon512* pair is used. As opposed to the cases explained above, we downgrade the KEM algorithm because Kyber1024 generates almost 800 bytes more ciphertext than Kyber512 does, which yields more packet transmissions when BLE is used. Regardless of the latency constraints, *Kyber512-Falcon512* pair is used if the IoT device operates on batteries.

9.8. Evaluation of computational overhead CPU time for different signatures and KEM

Since many IoT devices operate on battery power, evaluating their CPU overhead is essential for optimizing energy efficiency. To achieve accurate measurements, we utilized the Perf library [48], a high-precision benchmarking tool designed to profile and analyze CPU performance. This tool offers detailed insights into the computational cost linked to cryptographic operations on resource-constrained devices. As shown in Table 6, P256 has been included as a baseline metric to provide a conventional elliptic curve-based signature as a reference for comparison with post-quantum and hybrid cryptographic schemes. It also shows that P256 achieves lower CPU overhead compared to hybrid schemes, particularly when paired with other PQ signatures. Similarly, Falcon512 has been selected due to its efficient lattice-based signature scheme, which offers strong PQ security while maintaining relatively low computational overhead comparing when it is paired with hybrid certificate. Moreover, OCSP Stapling consistently increases CPU overhead, with an average increase of 17%–35%. In cases like P256/Dilithium2, along with hybrid KEM P256/Kyber512, which jumps from 33.66 ms using no certificate validation to 45.21 ms OCSP stapling. Among hybrid signatures, P256/Falcon512 is the most efficient, whereas P256/S+SHA incurs the highest computational overhead, with P256/S+SHA-BLE (P521/Ky1024) reaching 109.71 ms, making it the most computationally expensive configuration. These results emphasize that OCSP Stapling, and larger key sizes introduce substantial overheads, critical considerations for resource-constrained environments. Optimizing key selection and validation methods is crucial to balancing security and performance in real-world deployments.

Table 6
Computational overhead CPU time for different signatures and KEM (ms).

Signature	OCSP Stapling		
	X25519	P256/Ky512	P521/Ky1024
P256	22.34	24.52	76.12
Falcon512	26.46	28.07	79.51
Dilithium2	29.56	34.31	82.60
S+ SHA	48.82	50.02	98.73
P256/Fal512	29.85	34.89	86.52
P256/Dil2	37.13	45.21	91.26
P256/S+SHA	55.66	62.30	109.71
	No Cert. Validation		
	X25519	P256/Ky512	P521/Ky1024
P256	18.53	22.54	70.81
Falcon512	20.98	25.46	73.10
Dilithium2	24.44	27.71	78.66
S+ SHA	35.12	34.34	89.91
P256/Fal512	21.45	24.15	78.00
P256/Dil2	27.34	33.66	80.63
P256/S+SHA	42.23	51.09	96.76

Table 7
Unique memory footprint for different signatures and KEM (KB).

Signature	OCSP Stapling		
	X25519	P256/Ky512	P521/Ky1024
P256	3168	3332	3344
Falcon512	3516	3628	3660
Dilithium2	3592	3656	3684
S+ SHA	3660	3708	3732
P256/Fal512	3600	3676	3696
P256/Dil2	3624	3684	3712
P256/S+SHA	3692	3752	3800
	No Cert. Validation		
	X25519	P256/Ky512	P521/Ky1024
P256	3124	3308	3328
Falcon512	3488	3584	3636
Dilithium2	3532	3624	3660
S+ SHA	3628	3680	3700
P256/Fal512	3572	3632	3664
P256/Dil2	3596	3664	3688
P256/S+SHA	3652	3668	3736

9.9. Evaluation of unique memory footprint for different signatures and KEM

The results in Table 7 indicate that P256 has the lowest memory footprint among the evaluated schemes. Falcon512 would be following with the lowest memory footprint compared to PQ as well as hybrid PQ signatures. OCSP stapling introduces a measurable overhead and increases memory footprint across all configurations, with an average overhead of approximately 1.00% compared to no certificate validation. For example, P256/Falcon512, along with ECDH, sees a memory increase from 3572 ms to 3600 ms, which is an increase of 0.78%, while P256/S+SHA with hybrid KEM (P521/Kyber1024) experiences the highest OCSP overhead at 1.71%. Larger key sizes such as P521/Ky1024 require up to 3.12% more memory than ECDH. Moreover, comparing hybrid signatures, P256/S+SHA consistently consumes the most memory, whereas P256/Falcon512 is the most memory-efficient across all configurations.

9.10. Energy consumption

Since numerous IoT devices rely on batteries, it is critical to examine their energy usage. To address this, we used a MakerHawk USB multimeter device capable of precisely measuring and displaying the power usage of a Raspberry Pi as it operates [49]. A figure of this setup is shown in Fig. 10. We report the Raspberry Pi's energy usage during idle periods and when initiating a TLS connection with the web server. Various signature schemes and KEM algorithms were explored. The idle stage of the RPi was 5.34 V (V) and from 0.465 to 0.473 Ampere (A).

The energy consumption of an RPi while idling is between 2.48 and 2.52 Watts (W), which is calculated as Voltage (V) multiplied by the Current (A).

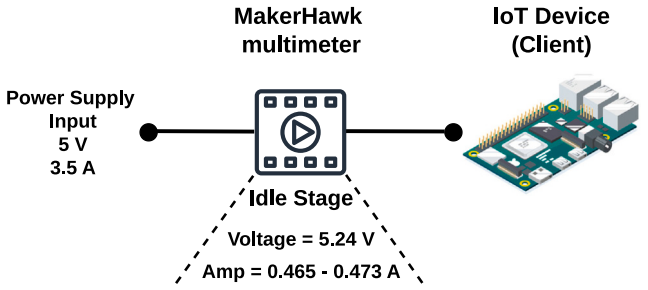

Fig. 10. Energy consumption setup for the Raspberry Pi using the MakerHawk multimeter.

Table 8
Energy consumption of TLS handshake with a single and mutual authentication (Watts).

Signature Scheme	Single Authentication		
	ECDH	Kyber512	Kyber1024
ECDSA-WF	2.85	3.00	3.06
ECDSA-BLE	2.73	2.80	2.81
RSA-WF	2.97	3.03	3.08
RSA-BLE	2.78	2.83	2.88
Falcon512-WF	3.00	3.02	3.08
Falcon512-BLE	2.85	2.82	2.92
Dilithium2-WF	3.01	3.01	3.12
Dilithium2-BLE	2.88	2.87	2.94
S+ SHA-WF	2.99	3.05	3.13
S+ SHA-BLE	2.91	2.95	3.02
	Mutual Authentication		
	ECDH	Kyber512	Kyber1024
ECDSA-WF	2.93	3.04	3.10
ECDSA-BLE	2.80	2.84	2.87
RSA-WF	3.01	3.08	3.10
RSA-BLE	2.84	2.89	2.96
Falcon512-WF	3.04	3.09	3.12
Faclon512-BLE	2.89	2.96	3.07
Dilithium2-WF	3.04	3.14	3.18
Dilithium2-BLE	2.98	3.09	3.01
S+ SHA-WF	3.33	3.47	3.58
S+ SHA-BLE	3.26	3.40	3.52

Therefore, as shown in Table 8, we tested various signature schemes and KEM algorithms with different authentication methods, such as single and mutual authentication. For example, using a Wi-Fi interface with Dilithium2/ Kyber1024, a single TLS connection only costs around 3.12 W; then, energy will drop to its idle state after establishing a secure connection using PQ, which indicates that the additional energy overhead associated with our testing is insignificant. We also tested different validation methods, such as CRLs and OCSP stapling, and the results were similar to those since most of the validation was done on the cloud and the server side. Overall, the energy consumption of IoT devices when establishing TLS connections with PQ does not bring additional energy overhead on the device, regardless of the chosen signature and KEM algorithms. This is important as it provides freedom for choosing the parameters in terms of energy metrics.

The other important outcome of these experiments is that, as shown for Falcon512/Kyber512, the energy overhead is lower when using BLE since it uses fewer resources than the Wi-Fi interface.

10. Discussion on different hardware architectures

PQC has been considered a critical subject of comprehensive academic exploration for a considerable time, mainly due to the increasing risks of quantum computing introduced to established cryptographic practices. Therefore, testing the feasibility of PQC on constrained hardware has been an essential focus of recent studies. Researchers have evaluated PQC implementations on various embedded platforms used in security-critical applications, including ARM Cortex M4, ESP32 microcontroller, and FPGA-based architectures such as Kintex7 [50]. Moreover, the performance of standardized PQC algorithms, including hash-based schemes as well as lattice-based such as Falcon, Dilithium, and Kyber, has been tested for applications such as TLS and secure firmware updates. The results indicate that Falcon's certification was faster than Dilithium, and SHINCS+ had the slowest signing process and larger signatures, making it less suitable for embedded TLS use cases. For memory usage, PQ TLS required 40 KB of RAM, compared to around 30 times less for conventional elliptic-curve TLS implementations. For secure firmware updates, Falcon had the smallest signatures and fastest certification but had higher implementation costs. As for Dilithium, it requires higher memory usage. On

the other hand, another study [51], evaluates PQC implementations on Arduino Nano RP2040 Connect (ARM Cortex M0+) and TinyPICO (Xtensa LX7), demonstrating the feasibility and impact of transitioning TLS 1.3 to PQ security. Moreover, performance evaluation highlights that PQC significantly increases bandwidth consumption and execution time, with PQ certificate chains being 10 times larger than elliptic curve-based ones. Full TLS handshake times on the Nano RP2040 increased from 584 ms (ECC-based) to 771 ms (PQ) and 2006 ms (hybrid), while resumption handshakes showed a similar trend. Memory requirements also increased from 16 KB (ECC) to 58 KB (PQC), making optimizations essential for IoT deployments.

11. Conclusion

In this paper, we investigated the viability of the quantum-resistant cryptographic algorithms that have recently been standardized by the NIST in a wireless IoT network leveraging Wi-Fi and BLE which are widely used standards in consumer applications. The idea was to compare the performance of the algorithms over TLS in two ways and offer recommendations. We did not only measure the mere performance of the PQ digital signature schemes using ECDH for key exchange, but also assessed its performance when combined with the PQ KEM schemes in order to determine their efficiency over a BLE connection. We offered a dynamic switching approach among the existing PQ protocol suites of an IoT device to offer the optimal performance for TLS handshakes.

In accordance with the experiment results, the Falcon512 digital signature & Kyber512 KEM combination outperforms the others in a BLE network, even when mutual authentication is needed. Moreover, Dilithium2 and Falcon512 proved to be the best option considering the performance and the security measures over Wi-Fi. Hybrid algorithms showed comparable results on Wi-Fi. However, on BLE, the figures increased more than 50% from P256/Falcon512 to P256/Dilithium2. Overall, Falcon512 and its hybrid variants showed the best performance among all the digital signature algorithms. Therefore, as long as the IoT device hardware supports floating-point arithmetic Falcon512 should be preferred. Otherwise, Dilithium2 is the best alternative. At the initial phase of the migration to PQC, hybrid algorithms should be considered to guarantee at least a security level of classical counterpart of the hybrid algorithm in case any vulnerabilities for the quantum-resistant counterpart are revealed. Although the certificate validation method using CRLs performs better than that using OCSP stapling, we recommend using the latter since it both provides up-to-date certificate status information and does not require additional memory space to maintain CRLs. Also, the communication overhead due to the OCSP requests can be eliminated by caching the response. The experiments for our dynamic PQ KEM approach showed that it incurs considerable computational delay. Nonetheless, it remains within acceptable time limits even when mutual authentication is required. Finally, the impact of using PQ algorithms for TLS on energy consumption does not change across the board.

Test results demonstrated the need for optimizing PQC algorithms for low power and low bandwidth IoT devices without being targeted by any side-channel attack. Moreover, the TLS handshake protocol introduces excessive amount of message overhead. Therefore, an alternative key exchange mechanism such as Ephemeral Diffie–Hellman Over COSE (EDHOC) should be considered in the future work. Furthermore, integrating PQC with blockchain-based IoT security models should be studied in order to render the blockchain robust against quantum attacks. Lastly, to increase diversity in the experiments, low-end IoT devices can be emulated using development boards running innovative and small-scale operating systems such as RIOT [52], Mbed OS [53], Contiki-NG [54], FreeRTOS [55], and Zephyr [56].

CRediT authorship contribution statement

Yacoub Hanna: Writing – review & editing, Validation, Resources, Data curation. **Jessica Bozhko:** Writing – review & editing, Validation. **Samet Tonyali:** Writing – review & editing, Visualization, Validation, Supervision, Resources. **Ricardo Harrilal-Parchment:** Writing – review & editing, Resources. **Mumin Cebe:** Writing – review & editing, Visualization, Validation. **Kemal Akkaya:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

In this research, Yacoub Hanna is supported by USAID under grant No. 1004242-S21-35022-00. Also, it was funded by a National Centers of Academic Excellence in Cybersecurity grant (H98230-21-1-0324 (NCAE-C-002-2021)), which is part of the US National Security Agency and National Science Foundation under the grant No. 2147196.

Data availability

I have shared the link of the code in the manuscript paper.

References

[1] K. Shibagaki, Achieving Scalability: The key to future quantum computers, 2020, https://www.mri.co.jp/en/50th/columns/quantum/no02/, [Online; Accessed 11 August 2023].
[2] S.S. Gill, A. Kumar, H. Singh, M. Singh, K. Kaur, M. Usman, R. Buyya, Quantum computing: A taxonomy, systematic review and future directions, Softw.: Pr. Exp. 52 (1) (2022) 66–114.
[3] P.W. Shor, Algorithms for quantum computation: discrete logarithms and factoring, in: Proceedings 35th Annual Symposium on Foundations of Computer Science, Ieee, 1994, pp. 124–134.
[4] P.W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, SIAM Rev. 41 (2) (1999) 303–332.
[5] D.J. Bernstein, A.K. Lenstra, A general number field sieve implementation, in: The Development of the Number Field Sieve, Springer, 2006, pp. 103–126.
[6] National Institute of Standards & Technology, Post-Quantum Cryptography, 2016, https://csrc.nist.gov/projects/post-quantum-cryptography, [Online; Accessed 29 August 2024].
[7] NIST, NIST Releases First 3 Finalized Post-Quantum Encryption Standards, 2024, https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards, [Online; Accessed 29 August 2024].
[8] D. Stebila, M. Mosca, Post-quantum key exchange for the internet and the open quantum safe project, in: International Conference on Selected Areas in Cryptography, Springer, 2016, pp. 14–37, URL https://www.mdpi.com/1424-8220/22/2/489/htm.
[9] J. Nieminen, T. Savolainen, M. Isomaki, B. Patil, Z. Shelby, C. Gomez, RFC 7668-IPv6 over bluetooth low energy, IETF: Fremont, CA, USA (2015).
[10] Minew, G1 IoT Gateway, 2021, https://www.minew.com/product/g1-iot-gateway/, [Online; Accessed 21 August 2023].
[11] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies, 2020, pp. 149–156.
[12] K. Bürstinghaus-Steinbach, C. Krauß, R. Niederhagen, M. Schneider, Post-quantum tls on embedded systems: Integrating and evaluating kyber and sphincs+ with mbed tls, in: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, 2020, pp. 841–852.
[13] D.S. Gupta, S.H. Islam, M.S. Obaidat, A. Karati, B. Sadoun, LAAC: Lightweight lattice-based authentication and access control protocol for E-health systems in IoT environments, IEEE Syst. J. 15 (3) (2020) 3620–3627.
[14] M. Adeli, N. Bagheri, H.R. Maimani, S. Kumari, J.J. Rodrigues, A post-quantum compliant authentication scheme for IoT healthcare systems, IEEE Internet Things J. (2023).
[15] G. Banegas, K. Zandberg, E. Baccelli, A. Herrmann, B. Smith, Quantum-resistant software update security on low-power networked embedded devices, in: International Conference on Applied Cryptography and Network Security, Springer, 2022, pp. 872–891.
[16] S. Sarıbaş, S. Tonyalı, Performance evaluation of TLS 1.3 handshake on resource-constrained devices using NIST's third round post-quantum key encapsulation mechanisms and digital signatures, in: 2022 7th International Conference on Computer Science and Engineering, UBMK, IEEE, 2022, pp. 294–299.
[17] G. Tasopoulos, J. Li, A.P. Fournaris, R.K. Zhao, A. Sakzad, R. Steinfeld, Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems, in: International Conference on Information Security Practice and Experience, Springer, 2022, pp. 432–451.
[18] G. Tasopoulos, C. Dimopoulos, A.P. Fournaris, R.K. Zhao, A. Sakzad, R. Steinfeld, Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices, Cryptol. EPrint Arch. (2023).
[19] C. McLoughlin, C. Gritti, J. Samandari, Full post-quantum datagram TLS handshake in the Internet of Things, in: International Conference on Codes, Cryptology, and Information Security, Springer, 2023, pp. 57–76.
[20] Y. Hanna, D. Pineda, M. Veksler, M. Paudel, K. Akkaya, M. Anastasova, R. Azarderakhsh, Integrating post-quantum TLS into the control plane of 5G networks, in: 2024 IEEE International Performance, Computing, and Communications Conference, IPCCC, IEEE, 2024, pp. 1–8.
[21] A. Mutlugun, Y. Hanna, K. Akkaya, Performance evaluation of quantum-resistant IKEv2 protocol for satellite networking environments, in: 2024 IEEE Virtual Conference on Communications, VCC, IEEE, 2024, pp. 1–7.
[22] K. Sabanci, Exploring Post-Quantum Cryptographic Schemes for TLS in 5G Nb-IoT: Feasibility and Recommendations, Master s thesis, Marquette University, 2023.
[23] J. Bozhko, Y. Hanna, R. Harrilal-Parchment, S. Tonyali, K. Akkaya, Performance evaluation of quantum-resistant TLS for consumer IoT devices, in: 2023 IEEE 20th Consumer Communications & Networking Conference, CCNC, IEEE, 2023, pp. 230–235.
[24] C. Ugwuishiwu, U. Orji, C. Ugwu, C. Asogwa, An overview of quantum cryptography and shor's algorithm, Int. J. Adv. Trends Comput. Sci. Eng 9 (5) (2020).
[25] National Institute of Standards & Technology, NIST announces first four quantum-resistant cryptographic algorithms, 2022, URL https://www.nist.gov/news-events/news/2022/07/nist-announces-first-four-quantum-resistant-cryptographic-algorithms.
[26] OQS-OpenSSL_1_1_1, 2022, URL https://github.com/open-quantum-safe/openssl/blob/OQS-OpenSSL_1_1_1-stable/README.md.
[27] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J.M. Schanck, P. Schwabe, G. Seiler, D. Stehlé, CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM, in: 2018 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2018, pp. 353–367.
[28] OQS Benchmarking, URL https://openquantumsafe.org/benchmarking/, Open Quantum Safe Website.
[29] OQS Algorithms, URL https://openquantumsafe.org/liboqs/algorithms/, Open Quantum Safe Website.
[30] C. Ekechukwu, D. Lindskog, R. Ruhl, A notary extension for the online certificate status protocol, in: 2013 International Conference on Social Computing, IEEE, 2013, pp. 1016–1021.
[31] P. Hallam-Baker, X. 509v3 Transport Layer Security (tls) Feature Extension, Tech. rep, 2015.
[32] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, D. Cooper, Transition to Post-Quantum Cryptography Standards, Tech. rep, National Institute of Standards and Technology, 2024.
[33] C.J. Hoofnagle, B. Van Der Sloot, F.Z. Borgesius, The European union general data protection regulation: what it is and what it means, Inf. Commun. Technol. Law 28 (1) (2019) 65–98.
[34] I.O. for Standardization, Information Security, Cybersecurity and Privacy Protection-Information Security Management Systems-Requirements, ISO, 2022.
[35] D.J. Bernstein, Curve25519: New diffie-hellman speed records, in: M. Yung, Y. Dodis, A. Kiayias, T. Malkin (Eds.), Public Key Cryptography - PKC 2006, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 207–228.
[36] R. Salles, R. Farias, TLS protocol analysis using IoTST—An IoT benchmark based on scheduler traces, Sensors 23 (5) (2023) 2538.
[37] M. Kerrisk, The linux programming interface: a linux and UNIX system programming handbook, No Starch Press, 2010.
[38] R. Petersen, R. Petersen, Network configuration, Begin. Fedora Deskt.: Fedora 28 Ed. (2018) 549–581.
[39] A. Dutta, E. Karagoz, E. Persichetti, P. Sanal, Polynomial inversion algorithms in constant time for post-quantum cryptography, in: International Conference on Cryptology in India, Springer, 2024, pp. 237–256.
[40] S. Kundu, A. Karmakar, I. Verbauwhede, On the masking-friendly designs for post-quantum cryptography, in: International Conference on Security, Privacy, and Applied Cryptography Engineering, Springer, 2023, pp. 162–184.
[41] M. Azouaoui, Y. Kuzovkova, T. Schneider, C. van Vredendaal, Post-quantum authenticated encryption against chosen-ciphertext side-channel attacks, IACR Trans. Cryptogr. Hardw. Embed. Syst. (2022) 372–396.
[42] A. Orlenko, Khvzak/Bluez-Tools: A set of tools to manage bluetooth devices for linux. URL https://github.com/khvzak/bluez-tools.

[43] OCSP Caching. URL https://www.ssl.com/article/page-load-optimization-ocsp-stapling/.
[44] TCPDUMP tool, 2024, URL https://github.com/the-tcpdump-group/tcpdump/tree/tcpdump-4.99.5.
[45] Wireshark website, 2023, URL https://www.wireshark.org/.
[46] D. Moody, Parameter selection for the selected algorithms, 2022.
[47] P. Kampanakis, D. Sikeridis, Two post-quantum signature use-cases: Non-issues, challenges and potential solutions, in: Proceedings of the 7th ETSI/IQC Quantum Safe Cryptography Workshop, Seattle, WA, USA, vol. 3, 2019.
[48] Perf Library, URL https://github.com/torvalds/linux/blob/master/tools/perf/perf.c, CPU Overhead Perf Library.
[49] MakerHawk Type-C USB Tester Voltmeter Meter 0-5A 4-30V USB Multimeter. URL https://www.makerfocus.com/products/makerhawk-type-c-usb-tester-voltmeter-meter-0-5a-4-30v-usb-multimeter-voltage-and-current-tester?variant=42382844559598.
[50] M. Pursche, N. Puch, S.N. Peters, M.P. Heinl, SoK: The engineer's guide to post-quantum cryptography for embedded devices, Cryptol. EPrint Arch. (2024).
[51] M. Scott, On TLS for the internet of things, in a post quantum world, Cryptol. EPrint Arch. (2023).
[52] E. Baccelli, C. Gündoğan, O. Hahm, P. Kietzmann, M.S. Lenders, H. Petersen, K. Schleiser, T.C. Schmidt, M. Wählisch, RIOT: An open source operating system for low-end embedded devices in the IoT, IEEE Internet Things J. 5 (6) (2018) 4428–4440.
[53] N. Strelkov, V. Krutskikh, E. Shalimova, Programming STM32 nucleo platform for IoT education using STM32duino and mbed OS, in: 2022 VI International Conference on Information Technologies in Engineering Education (Inforino), IEEE, 2022, pp. 1–6.
[54] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, N. Tsiftes, The Contiki-NG open source operating system for next generation IoT devices, SoftwareX 18 (2022) 101089.
[55] R. Barry, et al., FreeRTOS, Internet, Oct 4 (2008).
[56] Y.-k. Lee, et al., Implementation of TLS and DTLS on Zephyr OS for IoT devices, in: 2018 International Conference on Information and Communication Technology Convergence, ICTC, IEEE, 2018, pp. 1292–1294.